

Lab2实验报告

思考题

Thinking 2.1

都是虚拟地址

Thinking 2.2

使用宏可以不需要对变量类型进行进行检查，可以适用于不同数据结构。

双向链表可以在已知数据前后插入，单向链表和循环链表只能在数据之后插入；双向链表可以直接删除已知数据，而单向链表和循环链表需要遍历得到前项数据才能删除。

Thinking 2.3

```
C:
struct Page_list{
    struct {
        struct {
            struct Page *le_next;
            struct Page **le_prev;
        } pp_link;
        u_short pp_ref;
    } * lh_first;
}
```

观察 `include/queue.h` 中数据结构的使用，由 `(listelm)->field.le_prev` 可以发现 `listelm` 后用 `->` 说明 `lh_first` 是指针，`field` 后用 `.` 说明 `pp_link` 不是指针。

Thinking 2.4

`boot_pgdir_walk` 在 `boot_map_segment` 中被调用，`boot_map_segment` 在为 `Page` 结构体和 `Env` 结构体进行映射时被调用。

ASID 为 6 位，故可同时容纳 64 个不同地址空间。

Thinking 2.5

MMU 在将虚拟地址转换成物理地址时首先去 TLB 中找合适 entry，如果能找到，则立即返回物理地址。因为 process 的页表具有私密性，所以所以 TLB 中的 entry 需要保证只有本 process 可以访问。MMU 在 TLB 中查询时需要判断这个 ASID 和当前进程的 ASID 是否一致，只有一致才证明这条 entry 当前 process 有权限访问。

Thinking 2.6

tlb_invalidate调用tlb_out。

tlb_invalidate作用：作废tlb，防止因页表内容变化而访问到错误物理页面。

```
LEAF(tlb_out)
//1: j 1b
nop
    mfc0      k1,CP0_ENTRYHI    //把EntryHi的值存到k1寄存器
    mtc0      a0,CP0_ENTRYHI    //把传入的参数key存到EntryHi
    nop
    tlbp                                //根据key找到TLB中对应表项，将索引存入index
    // insert tlbp or tlbwi
    nop
    nop
    nop
    nop
    mfc0      k0,CP0_INDEX      //把索引后的index存到k0
    bltz      k0,NOFOUND        //若k0小于0则没有匹配的表项，跳转NOFOUND
    nop
    mtc0      zero,CP0_ENTRYHI  //清空EntryHi
    mtc0      zero,CP0_ENTRYLO0 //清空EntryLo
    nop
    tlbwi                                //以 Index 寄存器中的值为索引，清空表项
    // insert tlbp or tlbwi
NOFOUND:
    mtc0      k1,CP0_ENTRYHI    //还原EntryHi的值

j    ra
nop
END(tlb_out)
```

Thinking 2.7

$PTbase + (PTbase \gg 12) * 4 + (PTbase \gg 24) * 4$

Thinking 2.8

x86根据两种不同的运行模式，有三种不同的内存管理方式：

1.实模式下，通过偏移地址加段寄存器值直接访问物理地址

2.保护模式下有两种内存管理方式：

段式管理：使用段寄存器作为索引，寻找GDT中相应的表项，获得该段的基址，逻辑地址作为偏移与基址相加，就得到物理地址。

地址分为两层：逻辑地址、物理地址

页式管理：首先通过GDT将逻辑地址转换成线性地址，在通过页目录表和页表将线性地址转换成物理地址。这一方式与MIPS页式管理内存相似。

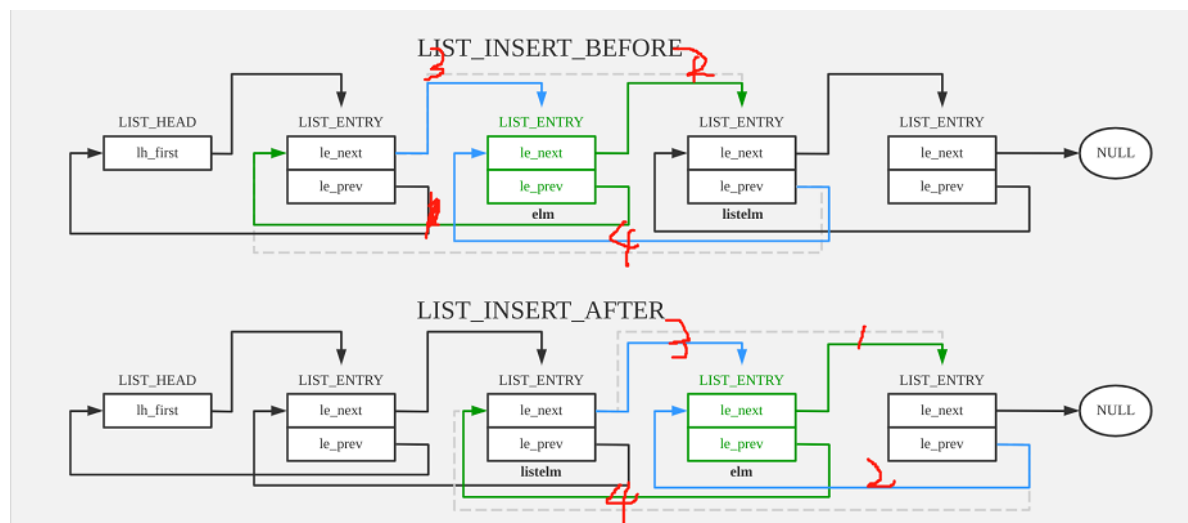
地址分为三层：逻辑地址、线性地址、物理地址。此处的逻辑地址相当于段式管理的物理地址。

练习题

Exe 2.1

分别对四个变量赋值即可，从Lab1内存布局得知物理地址最大值是0x1FFFFFFF

Exe 2.2



这部分主要考察数据结构链表插入的方法，`LIST_INSERT_AFTER`可以模仿`LIST_INSERT_BEFORE`实现，要注意这部分的数据结构比较复杂，`le_prev`是指针的指针。

`LIST_INSERT_TAIL`要先从`head`开始遍历链表到尾后插入。这部分较复杂，要很小心指针的变换和赋值用法。

Exe 2.3-2.5

根据框架和提示一步步完成即可，注意`page_free_list`类型是`struct Page_list`而不是指针，在宏函数中调用时要用`&`取地址。

Exe 2.6-2.7

注意`alloc`返回的是虚拟地址，而页表项需要的是物理地址，需要用`PADDR`进行转换。

Exe 2.8

模仿`boot`版本即可，但注意分配物理页面后要使其`pp_ref++`。

体会与感想

本次实验难度适中，第一部分主要难度在于理解链表的数据结构构造和理解链表宏的使用方法，第二部分难度在于理解虚拟地址向物理地址转换的方式，其中地址的计算非常绕，需要冷静下来理清楚思路，搞清楚地址的对应。整个Lab2耗时大约10h。